

How inkless Writes a PDF

A tour of the format, by hand, with no dependencies

Sankalp

2026-01-01

Contents

What a PDF actually is	4
The eight object types	4
Names are not strings	4
Numbers have no exponent form	4
Strings and the escaping trap	5
Testing the round trip	5
Why the cross-reference table is the risky part	5
Why the file is built in one buffer	6
The trailer	6
Fonts without font files	6
The consequence for a zero dependency tool	6
Which fourteen	6
Measuring a string	6
What cannot be drawn	7
The placeholder policy	7
Turning Markdown into boxes	7
Scanning without regular expressions	7
Offsets on every node	7
Byte offsets and character offsets	7
Emphasis is harder than it looks	8
Unmatched delimiters	8
Greedy line breaking	8
Justification	8
Pagination and the details that show	8
Widows and orphans	9
Keeping a heading with its text	9
Things that must break	9
The table of contents problem	9
The circular dependency	9
How inkless closes the circle	10
Destinations, not just page numbers	10
Clicking versus scrolling	10
Determinism	10
What breaks reproducibility	10
The document identifier	11
Compression is deterministic too	11
Honest limits	11

What is missing	11
What it is slower than	11
Where to look first	12

What a PDF actually is

A PDF file is not a binary blob in the way a JPEG is. It is closer to a text file with a strict internal filing system. Open one in a text editor and the first line reads `%PDF-1.7`, and much of what follows is readable ASCII. Understanding that is the difference between reaching for a library and writing the format yourself, which is what this program does.

The file has four parts in order: a header, a body of numbered objects, a cross-reference table, and a trailer. The header is two lines, the second of which is a comment containing bytes above 127 so that any tool moving the file treats it as binary rather than helpfully rewriting its line endings.

The eight object types

Everything inside a PDF is built from eight kinds of value. There are no others, and once you can write these you can write any PDF.

Type	Written as	Note
null	<code>null</code>	rarely needed
boolean	<code>true false</code>	watch Python, where <code>bool</code> is an <code>int</code>
number	<code>12 3.5</code>	no exponent notation is permitted
string	<code>(literal)</code> or <code><hex></code>	escaping matters, see below
name	<code>/Type</code>	a token, not a string
array	<code>[1 2 3]</code>	whitespace separated
dictionary	<code><< /Key value >></code>	keys are names
stream	a dictionary then raw bytes	carries <code>/Length</code>

A ninth construct, the indirect object, wraps any of these in `N 0 obj` and `endobj` so that it can be referred to elsewhere as `N 0 R`. That is the entire object model.

Names are not strings

A name such as `/Type` is a single token, not a quoted string. Any character outside a small safe set is written as a hash followed by two hex digits, so a name containing a space becomes `/with#20space`. Getting this wrong produces a file that looks fine and fails to open.

Numbers have no exponent form

A PDF number is digits, an optional sign and an optional decimal point. That is all. Python's `repr` of a float is free to produce exponent notation, so every number here goes through a formatter that emits fixed notation and trims the trailing zeros.

Strings and the escaping trap

Inside a literal string three characters must be escaped with a backslash: the opening parenthesis, the closing parenthesis, and the backslash itself. Miss one and the string swallows the rest of the object, and every reader rejects the file or renders nonsense.

This document contains (parentheses) and a backslash \ on purpose, so that opening it is itself a test of the escaping. inkless goes further and writes every byte outside printable ASCII as a three digit octal escape, which keeps the whole output file pure ASCII.

Testing the round trip

Encoding is only half of it. The test suite decodes the emitted strings back with a separate decoder written independently of the encoder, so a bug shared between an encoder and its own inverse cannot hide.

```
def encode_literal_string(data: bytes) -> bytes:
    """Escape bytes for a PDF literal string, including the parentheses."""
    out = bytearray(b"(")
    for byte in data:
        if byte in (0x5C, 0x28, 0x29):
            out.append(0x5C)
            out.append(byte)
        elif 0x20 <= byte <= 0x7E:
            out.append(byte)
        else:
            out += b"\\%03o" % byte
    out += b")"
    return bytes(out)
```

Why the cross-reference table is the risky part

The cross-reference table lists the byte offset of every object in the file. Not an index, not a name: the literal number of bytes from the start of the file to the first character of that object's header.

Get an offset wrong by a single byte and every conforming reader rejects the whole document. There is no partial credit and no graceful degradation.

This forces a particular shape on the writer. You cannot stream a PDF out and fix it up later without seeking. inkless builds the entire file in one buffer and records each object's position as it is appended, then writes the table from those recorded positions.

Each entry is exactly twenty bytes: ten digits of offset, a space, five digits of generation number, a space, a one letter type flag, and a two byte line ending. Object zero is always the head of the free list with generation 65535.

Why the file is built in one buffer

Recording an offset means knowing how many bytes precede it, which means every object ahead of it has already been serialised. The writer therefore appends into a single buffer and notes each position as it goes.

The trailer

The trailer names the root object, the total object count, and the byte offset of the table itself. A reader starts at the end of the file, reads `startxref`, seeks to the table, and works backwards from there. That is why a PDF can be opened without reading the middle of it.

```
xref
0 7
0000000000 65535 f
0000000015 00000 n
0000000064 00000 n
```

Fonts without font files

The single most surprising thing about PDF is that a conforming reader is required to provide fourteen fonts itself. Helvetica, Times and Courier in four cuts each, plus Symbol and ZapfDingbats. A file that uses only those never has to embed a single glyph.

The consequence for a zero dependency tool

This is why inkless can typeset without shipping a font file and without reaching for a font library. It names `/Helvetica` in a font dictionary, sets `/Encoding /WinAnsiEncoding`, and the reader supplies the outlines.

The catch is that line breaking happens in the writer, long before any reader sees the file, so the writer still needs to know how wide every character is. Those numbers are published as part of the font metrics, in units of one thousandth of an em, and inkless hard-codes them.

Which fourteen

Helvetica, Times and Courier each come in four cuts: regular, bold, italic or oblique, and the two combined. That is twelve. Symbol and ZapfDingbats make fourteen. inkless uses the twelve text cuts and ignores the other two.

Measuring a string

The measurement itself is arithmetic. Take each character, look up its advance width, add them, multiply by the point size, divide by a thousand.

- Courier is monospaced, so every glyph advances 600
- Helvetica and Times need real per-character tables

- The oblique and italic cuts share the widths of their upright partners
- Accented letters carry the width of their unaccented base letter

That last point is worth dwelling on. In these fonts an accented glyph is composed from a base letter and a floating accent, so `Aacute` advances exactly as far as `A`. Deriving those widths that way is exact and avoids inventing numbers.

What cannot be drawn

`WinAnsiEncoding` covers Latin script and common punctuation. It does not cover Greek, Cyrillic, Chinese, Japanese, Korean, or emoji. Without embedding a font there is no way to draw those glyphs at all.

`inkless` does not pretend otherwise. Any character outside the encoding is replaced with a question mark and reported on standard error with the line and column where it appeared. Silently dropping characters would be worse than useless.

The placeholder policy

A substituted character is replaced with a question mark, never dropped, and each distinct offending character is reported once with the position of its first appearance. Reporting every occurrence would bury the signal.

Turning Markdown into boxes

Parsing is only the first half. A tree of headings and paragraphs still has to become ink at specific coordinates, and that is where the typesetting decisions live.

Scanning without regular expressions

The parser here scans character by character with index arithmetic. No regular expressions are involved anywhere in the program.

The reason is not purity. A regular expression matches a span and hands back the span; it does not naturally hand back the byte offset, line and column of every token inside it. Scanning by index gives those for free, and they are what turns a vague complaint into a caret under the exact character that caused it.

Offsets on every node

Every node in the tree carries the offset, line and column of the character that produced it. Retrofitting that later is miserable, so it is built in from the first line of the scanner.

Byte offsets and character offsets

They are not the same as soon as the document leaves ASCII. Both are carried, because a caret has to line up with characters while a byte offset is what a tool downstream will expect.

```
Warning: unclosed emphasis
some bold text here
    ^
at line 12, column 6
```

Emphasis is harder than it looks

The obvious approach pairs each asterisk with the next one. It falls over immediately. Consider ***bold italic***, or ***a*b*c*d*e***, or bold nested inside a link label, or `snake_case_identifiers` that must survive unmangled.

inkless uses a delimiter stack. Each run of asterisks or underscores is recorded with whether it can open emphasis, whether it can close it, and how long it is. Closers are then matched back to openers under the flanking rules, including the rule that a pair whose lengths sum to a multiple of three is rejected unless both are themselves multiples of three.

Unmatched delimiters

A delimiter that never finds a partner becomes literal text, and the first unmatched opener is reported with a caret under it. That is more useful than reporting the closer, because the opener is where the author's intent went astray.

Greedy line breaking

Given a font, a size and a width, the line breaker takes words until the next one will not fit and then starts a line. This is first-fit, and it is what most word processors do.

It is not what TeX does. Knuth-Plass breaking considers the paragraph as a whole and minimises total badness, which produces visibly better results, especially in justified text. It is out of scope here and the difference is real: some justified lines below will have looser word spacing than a proper total-fit algorithm would produce.

A word wider than the whole column is broken by width rather than allowed to overflow the margin. No hyphen is inserted, because there is no hyphenation dictionary and a hyphen would imply a syllable boundary that has not actually been checked.

Justification

Justified text distributes the slack between the words of a line, and never on the last line of a paragraph. Because the breaking is greedy rather than total-fit, some lines carry more slack than a better algorithm would leave them.

Pagination and the details that show

Anyone can stack lines until the page is full. The difference between output that looks typeset and output that looks dumped is in what happens at the boundary.

Widows and orphans

An orphan is the first line of a paragraph left alone at the foot of a page. A widow is the last line left alone at the head of the next one. Both look like mistakes to a reader who could not tell you why.

inkless expresses both with one mechanism. Every row can be marked as kept with the row after it, and the paginator refuses to end a page on a kept row, pushing the whole chain forward instead.

- The first line of a multi-line paragraph is kept with the next, so it cannot be stranded at the bottom
- The second to last line is kept with the next, so the last line cannot be stranded at the top
- A heading is kept with whatever follows it, so it never ends a page

A three line paragraph therefore moves as a unit, because there is no way to split it that does not strand something.

Keeping a heading with its text

A heading that is the last thing on a page is the most obvious pagination failure of all, and the same keep-with-next flag prevents it. The heading pulls itself, and the first lines of what follows, onto the next page.

Things that must break

Not everything can be kept together. A code listing longer than a page has to break, and its background panel has to appear on both pages. inkless paints the panel per row rather than as one rectangle behind the block, which makes that fall out of pagination rather than needing a repair pass.

The same trick handles the rule down the side of a blockquote that crosses a page boundary.

The table of contents problem

A contents page is where a naive layout engine gives itself away, and it is worth spelling out why.

The circular dependency

To print a contents page you need to know which page each heading is on. To know that you need the final pagination. But inserting the contents page shifts every page after it, which changes the numbers you were about to print.

One extra pass is the obvious fix and it is not sufficient. If the contents grows from one page to two during resolution, every number it printed on the first pass is now short by one.

How inkless closes the circle

The body is paginated exactly once, because nothing placed before it changes where its own breaks fall. Only the offset added to its page numbers is in question, and that offset is the length of the contents itself.

So the layout loop guesses one page, lays the contents out, and if it needed more, raises the guess and tries again. The process is monotone: a larger offset can only make a printed number wider, which can only narrow the space left for a title, which can only produce the same number of lines or more. Rising guesses cannot oscillate, so the loop settles.

If a pass ever comes out shorter than its own guess, the contents are padded so that the numbers already printed stay true, and a hard bound on the pass count stops any pathological document spinning forever.

Destinations, not just page numbers

An entry that jumps to the top of the right page is still wrong if the heading is two thirds of the way down it. Each destination here is an `/XYZ` target carrying the heading's actual vertical position, so the reader lands on the heading itself.

The same destinations feed the outline tree, which is what fills the sidebar. That tree is doubly linked: every item names its parent, its previous and next siblings, and its first and last child.

Clicking versus scrolling

Two navigation paths exist and both are worth having. The contents entries are link annotations on the page itself, and the outline is a tree the reader draws in its sidebar. They point at the same destinations.

Determinism

Two runs over the same input produce byte identical files. This is not an accident and it takes deliberate work.

What breaks reproducibility

The usual culprit is a timestamp. A document information dictionary wants a creation date, and the obvious thing to write is the current time, which guarantees that no two builds ever match.

inkless never reads the clock. The date comes from the front matter, from a command line flag, or from a fixed default. The test suite walks the syntax tree of the program and fails the build if any call to a clock function appears anywhere in it.

The second culprit is iteration order. Anything derived from a set, or from a hash that varies between runs, leaks into the output. Every mapping that reaches the file here is an ordered dictionary built in a fixed sequence.

The document identifier

A PDF trailer carries an `/ID` array, and most producers fill it from the time and the file name. inkless derives it from a digest of the objects themselves, so it is stable across runs while still changing whenever the content does.

Compression is deterministic too

Content streams are compressed, and the compressor is deterministic for a given build of the library, so repeated runs on one machine produce identical bytes.

Honest limits

Every tool has a shape, and pretending otherwise helps nobody.

What is missing

- No font embedding, so no Greek, Cyrillic, CJK or emoji. Anything outside WinAnsiEncoding becomes a placeholder and is reported with its position
- Greedy line breaking rather than Knuth-Plass, so justified text is looser
- No hyphenation. A word wider than its column is broken by width with no hyphen inserted, which is most visible in a narrow table column, where a header word can split as `Co lum` and `n four`
- No footnotes, no math, no raw HTML
- Setext headings, the underlined kind, are not supported
- Images are PNG only, 8 bit, RGB or RGBA, and not interlaced. Anything else, including palette and greyscale PNGs and any non-PNG file, warns with its source position and falls back to the alt text
- Only an image alone on its own line is embedded. An image inline in a sentence would have to sit on a text baseline and take part in line breaking, so those are shown as alt text
- Above character code 126 the width tables are partly derived rather than tabulated, and one shared pair of ascent and descent ratios places every baseline instead of per-font values
- A table does not repeat its header across a page break, and a single table row taller than the page overflows rather than splitting

What it is slower than

It is slower than reportlab, which is a mature C-backed library. For a document of this size the difference does not matter, and the tradeoff is deliberate: nothing to install, nothing to audit, nothing to break when a transitive dependency changes.

Where to look first

If the output is wrong, the cross-reference table is the place to start. It is the one part of the format where a one byte error destroys the whole file, and the test suite carries a small PDF reader specifically to check every offset against the object it claims to point at.